
LICENCIATURA EN CIENCIAS DE LA COMPUTACIÓN

Compiladores

Proyecto Compilador `tinyS`

Primer Semestre 2026

IMPORTANTE: Es total responsabilidad de los alumnos leer el presente documento y las consignas completas de cada etapa. No se responden consultas sobre consignas no leídas. Los nombres utilizados no son caprichosos por lo que si se les pide que las cosas se llamen de cierta forma es por algo.

1 Introducción

Como trabajo final de la asignatura compiladores los estudiantes desarrollarán un compilador para un lenguaje de programación reducido al que hemos llamado `tinyS`. Aunque es un lenguaje académico incluye características propias de los lenguajes de programación modernos. `tinyS` es **type safe**: se debe garantizar que los métodos sean aplicados a datos de tipo correcto. Mientras que el tipado estático impone una disciplina fuerte de programación en `tinyS`, también garantiza que no ocurran errores de tipo en tiempo de ejecución. `tinyS` es fuertemente tipado.

La descripción del lenguaje de programación `tinyS` se encuentra completamente desarrollada en el documento respectivo. Si faltara alguna especificación se tomará como referencia el comportamiento del lenguaje Java.

Aquí comenzaremos haciendo una breve descripción de las etapas por las que pasa el desarrollo de un compilador para un lenguaje de programación moderno y, en particular, el que deberán transitar a medida que avancen en el trabajo final. Se espera que desarrollen estas etapas para que puedan detectar errores propios de cada etapa del compilador de forma de no hacer el efecto “bola de nieve” en su proyecto. Asimismo, deberán entregar cada etapa por separado junto con el compilador completo para evaluarlo de forma de establecer si realizaron correctamente el desarrollo de cada etapa y en caso de encontrar errores significativos poder brindar una devolución previo al examen final.

Durante la cursada se espera que vayan realizando las etapas a medida que se vayan viendo los conceptos relacionados con cada una. Sin embargo, el equipo de cátedra no evaluará ninguna de ellas hasta finalizado el semestre, cuando el alumno regularice y esté por presentarse al examen final. Se debe entregar 15 días antes del día del examen final.

Como sabemos por el manual de `tinyS`, un código fuente a compilar en `tinyS` estará compuesto por un único archivo.

2 Partes del Proyecto Compilador (Resumen)

El desarrollo del compilador estará dividido en cinco etapas, la última es el compilador final completamente implementado y funcionando.

1. Análisis Léxico: En esta parte deberá implementar un *analizador léxico* que reciba como entrada un código fuente y retorne como salida la lista de tokens del mismo, especificando para cada token su lexema y el número de línea y columna en la cual se encuentra. Este programa deberá ser capaz de detectar y reportar en forma adecuada todos los errores que puedan identificarse en la etapa de análisis léxico.
2. Análisis Sintáctico: En esta etapa deberá implementar un analizador sintáctico descendente predictivo recursivo que reciba un código escrito con la sintaxis de **tinyS** y retorne un mensaje adecuado que indique si el código recibido es o no es sintácticamente correcto. En caso de existir errores, el analizador deberá reportar en forma adecuada el primer error que encuentre (ya sea léxico o sintáctico) y luego interrumpir la ejecución.
3. Análisis Semántico - Tablas de Símbolos y Chequeo de Declaraciones: A partir de la gramática de **tinyS** modificada en la etapa anterior, se especificará un *esquema de traducción* que permita crear las estructuras vinculadas a las tablas de símbolos, la cual almacenará información correspondiente a todas las entidades declaradas. Se deberá definir cómo es la forma de estas estructuras y la porción del análisis semántico que las utiliza para controlar que toda entidad en la tabla ha sido correctamente declarada. En esta etapa deberán construir una porción del analizador semántico que extiende las funcionalidades del analizador sintáctico desarrollado en la etapa anterior, retornando un mensaje adecuado que indique si en el código recibido todas las entidades han sido correctamente declaradas. Al igual que en la implementación de la etapa anterior, en caso de existir errores el analizador deberá reportar el primer error encontrado (ya sea léxico, sintáctico o semántico) e interrumpir la ejecución.
4. Análisis Semántico - Árboles Sintácticos Abstractos y Chequeo de Sentencias: Con la gramática de **tinyS** modificada en la etapa anterior, se especificará un esquema de traducción que permita crear las estructuras de los árboles sintácticos abstractos correspondientes a los bloques de código de programa entrante. Se deberá definir cómo es la forma de estas estructuras y cómo son utilizadas para realizar los distintos controles semánticos asociados a las sentencias de un programa **tinyS**. En esta etapa se entregará un analizador semántico completo de **tinyS** que extiende las funcionalidades de lo desarrollado en la etapa anterior, retornando un mensaje adecuado que indique si el código recibido es o no semánticamente correcto. Al igual que en la implementación de la etapa anterior, en caso de existir errores el analizador deberá reportar el primer error encontrado (ya sea léxico, sintáctico o semántico) e interrumpir la ejecución.
5. Generación de Código Intermedio - Compilador Completo + Documentación: A partir del árbol sintáctico abstracto desarrollado en la etapa anterior, deberá especificarse la generación de código intermedio para un programa **tinyS**. En esta etapa se entregará el compilador completo de **tinyS**, el cual extiende los desarrollos de las etapas anteriores, añadiendo la generación de código intermedio. De esta manera, en caso de recibir un programa **tinyS** correcto, el compilador generará un archivo de salida con el código intermedio asociado al programa. En caso de que el programa contenga errores léxicos, sintácticos o semánticos, se reportará el primero encontrado y no se generará archivo de salida. En el informe técnico de esta etapa deberá mostrarse cómo se modificaron las estructuras y controles del árbol sintáctico abstracto para reflejar la generación de código.

3 Sobre las entregas

- Cada alumno realizará el proyecto de manera **individual** o a lo sumo en grupo de **dos** (2) personas.
- El formato de codificación del proyecto debe seguir los estándares subidos en la plataforma. Se recomienda leerlos **antes** de comenzar a codificar. Se evalúa la claridad del código escrito. Puede seguir otros estándares siempre que sean LEGIBLES.

- La entrega de todo el código fuente y compilado junto con los casos de prueba se realizará a través de la plataforma 15 días antes de la fecha del final `APELLIDO_ALUMNO/APELLIDO1_-APELLIDO2.tinyS.zip`. No entregar links de github o similar no serán revisados.
- En cada etapa, se deberá incluir un directorio `/tc` donde se encuentren los archivos fuente que fueron utilizados por el/la alumno/a para probar el funcionamiento de la porción del compilador entregada, especificando en el informe qué técnicas de testing fueron empleadas. Una etapa que no posea al menos 10 (diez) archivos fuente construidos por el alumno o el grupo dentro de la carpeta `/tc` **no se evaluará** y no podrá presentarse a final.
- Es responsabilidad de los alumnos que los archivos posean el formato adecuado y no estén corruptos. De ser necesario, se deberá agregar a la documentación una sección en la que se detalle cualquier tipo de problema o limitación relacionada con el proyecto.
- **En caso de que se detecte plagio con años anteriores o copia entre grupos o alumnos, todos los involucrados serán plausibles de las sanciones que la Facultad y Universidad establezcan.**

3.1 Lenguaje utilizado para la implementación de tinyS

La implementación del proyecto deberá llevarse a cabo en **Java**. Aquellos que por cuestiones debidamente fundadas prefieran utilizar **python** no tendrán soporte de la cátedra puesto que el proyecto está completamente armando para que el diseño y desarrollo del proyecto en **Java** sea factible en el tiempo de cursado de la asignatura para aquellos que quieran avanzar durante la cursada. Se requiere que el proyecto se pueda ejecutar utilizando el *Java Runtime Environment*. Aquellos que utilicen **python** la versión a utilizar deberá ser 3.11 o superior. Deberán tomarse el tiempo para verificar si las máquinas del laboratorio están preparadas para ejecutar su código y modificarlo para poder rendir el examen final. Notificar a la cátedra lo antes posible si requiere la instalación de software adicional para las evaluaciones presenciales antes de dichas evaluaciones (1 semana de anticipación). Tanto para **java** como para **python** queda prohibida la utilización de paquetes específicos para análisis léxico, sintáctico, semántico y de generación de código. Todas las entregas deberán ejecutarse por línea de comandos según la especificación de cada etapa. Los trabajos que no respeten este formato no serán evaluados **sin excepciones**.

3.2 Sobre la entrega fuera de término

En caso de que el alumno se quiera presentar al examen final, este anotado y se presente el día y hora sin realizar la entrega previa a la fecha (15 días antes), el examen se considerará desaprobado con 2 (dos).

Sobre el software en general

- El compilador deberá estar implementado en **Java** o en **Python** (Bajo responsabilidad del grupo) y ser una aplicación que pueda utilizarse en *línea de comandos*. Se recomienda utilizar algún entorno de desarrollo integrado como Eclipse, netbeans o IntelliJ IDEA con MAVEN. La elección es totalmente responsabilidad del/la/le/las/los/las alumno/a/es/as/os/.
- La codificación deberá seguir el estándar de codificación subido en el aula abierta sin excepciones.

4 Sobre el Manual del Desarrollador

Se deberá presentar el manual técnico (del desarrollador) con las siguientes características.

- Deberá incluirse la documentación sobre las etapas de Análisis Léxico (definición del alfabeto de entrada, tokens, etc.), Análisis Sintáctico (la evolución en el procesamiento de la BNF de **tinyS**, el análisis acerca de si es **LL(1)** o no y ¿el por qué de su afirmación?), el EDT completo que

especifique las acciones para construir el Árbol Sintáctico Abstracto (AST), y la especificación de la Tabla de Símbolos (estructuras utilizadas, jerarquía de clases para representar las entidades del lenguaje, etc.), y lo referido a chequeo de declaraciones y chequeo de sentencias.

- La forma de ejecutar el tinyS compilador.
- Toda aquella información sobre cómo se administró el tiempo del proyecto.
- Toda información relevante para que otra persona pueda extender las funcionalidades del compilador.
- Recuerde que es importante que la documentación sea autocontenida. Toda documentación adicional deberá estar en una carpeta que se entregue dentro del archivo .zip a entregar por plataforma.
- En el manual del desarrollador debe constar la descripción de los escenarios y casos de pruebas diferenciados entre etapas.

Tenga en cuenta que cada etapa también tiene sus características que deben estar presentes en el Manual del Desarrollador.

A continuación se detalla cada etapa del proyecto en forma individual.

5 Analizador Léxico

- El analizador deberá estar implementado en **Java** o en **Python** (Bajo responsabilidad del grupo) y ser una aplicación que pueda utilizarse en *línea de comandos*. Se recomienda utilizar algún entorno de desarrollo integrado como Eclipse, netbeans o IntelliJ IDEA. La elección es totalmente responsabilidad del/la/le/las/los/las alumno/a/es/as/os/.
- La codificación deberá seguir el estándar de codificación subido en el aula abierta sin excepciones.
- La aplicación debe estar desarrollada en **dos módulos** claramente diferenciados. El primero será el *analizador léxico* propiamente dicho que devolverá **tokens** y será el encargado de leer el archivo de entrada. El segundo módulo será el *ejecutador* para esta entrega. El *ejecutador* deberá generar y presentar una lista de los tokens reconocidos a partir del archivo fuente analizado. Para cada token se deberá mostrar el lexema reconocido y el número de línea donde fue encontrado.

La lista de tokens por pantalla/archivo deberá tener el siguiente formato de encabezado exacto, sin excepción. Utilizar para los errores letra mayúscula sin acentos ni símbolos poco frecuentes:

CORRECTO: ANALISIS LEXICO

| TOKEN | LEXEMA | NÚMERO DE LÍNEA (NÚMERO DE COLUMNA) |

Luego del encabezado, los tokens se pondrán uno debajo del otro como figura en el siguiente ejemplo:

CORRECTO: ANALISIS LEXICO

| TOKEN | LEXEMA | NÚMERO DE LÍNEA (NÚMERO DE COLUMNA) |

| id | pepe | LINEA 1 (COLUMNA 1) |

- Se entregará el código fuente, todos los archivos de proyecto, todo aquello necesario para ver cómo fue codificada la entrega y una carpeta con los casos de prueba que utilizó para probar su software (los .s). Para cada caso de test debe documentarse **en el Manual del Desarrollador o como comentario en el archivo .s** si es exitoso o no, su propósito y el resultado esperado. Además se deberá incluir documentación acerca de la forma de uso del proyecto para poder volver a compilarla y probar su funcionamiento.

- La *implementación* deberá realizarse utilizando alguna de las técnicas vistas en clase (**sin utilizar** herramientas de generación automática ni paquetes del lenguajes de programación que se utilizó para la implementación).
- El *manejo de errores* del analizador léxico deberá realizarse utilizando excepciones. Deberá reportarse un mensaje adecuado a la naturaleza del error e indicar en qué número de línea del fuente `tinyS` se produjo el mismo. El formato en caso de error de lo que se deberá mostrar en el encabezado será:

```
ERROR: LEXICO
| NÚMERO DE LÍNEA (NÚMERO DE COLUMNA) | DESCRIPCION: |
```

Por ejemplo, en caso de que el analizador detectara un identificador con un formato no permitido se mostrará:

```
ERROR: LEXICO
| NÚMERO DE LÍNEA (NÚMERO DE COLUMNA) | DESCRIPCION: |
| LINEA 1 (COLUMNA 1) | IDENTIFICADOR NO VALIDO Ho1@|
```

- La política ante la detección de un error es **abortar** el análisis léxico. No se implementarán técnicas de recuperación ante errores.
- Para simplificar la implementación, las palabras reservadas pueden manejarse mediante una tabla de identificadores como se presentó en clase.

La sintaxis de invocación deberá respetar la siguiente convención sin excepción:

```
java -jar etapa1.jar <ARCHIVO_FUENTE> [<ARCHIVO_SALIDA>]
```

Por ejemplo:

```
>java -jar etapa1.jar prueba.s prueba.txt. En el archivo prueba.txt se guarda la salida.
>java -jar etapa1.jar prueba.s. La salida se imprime por pantalla.
```

donde `etapa1.jar` es el nombre del ejecutable (obligatorio si se le coloca otro nombre la entrega será desaprobada), `<ARCHIVO_FUENTE>` es el archivo a analizar léxicamente (el usuario deberá poder elegir cuál archivo desea analizar) y `<ARCHIVO_SALIDA>` es un parámetro opcional que -de estar presente- especifica el archivo de salida donde se volcará el listado de tokens, lexemas y el número de línea generado, siguiendo el mismo formato presentado en pantalla. Si se omite este parámetro, el listado será presentado en pantalla.

Para aquellos que implementen en `Python` la sintaxis será:

```
python -m tinyS.etapa1 <ARCHIVO_FUENTE> [<ARCHIVO_SALIDA>]
```

Sobre el para la parte de Análisis Léxico

- El Manual del Desarrollador deberá seguir las pautas generales de presentación subidas en el aula abierta.
- El Manual deberá indicar de forma clara cómo compilar el código fuente entregado y cómo ejecutarlo una vez compilado.
- En el Manual deberá constar el **alfabeto de entrada** utilizado para `tinyS`.

- Deben especificarse los **tokens reconocidos** por el analizador léxico, cada uno acompañado de la *expresión regular* que lo define.
- Incluir los errores léxicos que el analizador puede de detectar.
- Deberá incluirse la documentación usual, propia de un proyecto de software (decisiones de diseño, clases utilizadas, etc.). Esta documentación debe estar en formato .pdf. Se considerará muy valioso que el código esté documentado (doxygen o javadoc o etc.) y deberá estar especificado en la documentación entregada qué se utilizó.

6 Analizador Sintáctico

- La política ante la detección de un error es **abortar** el análisis sintáctico. No se implementarán técnicas de recuperación ante errores.
- La sintaxis de invocación para aquellos que utilicen **java** deberán respetar la siguiente convención sin excepción:

```
java -jar etapa2.jar <ARCHIVO_FUENTE>
```

donde <ARCHIVO_FUENTE> es el nombre de un archivo a analizar sintácticamente. Por ejemplo,
>java -jar etapa2.jar caso_exitoso_01.s

Con respecto a aquellos que utilicen **python** será:

```
python -m tinyS.etapa2 <ARCHIVO_FUENTE>
```

- La salida del analizador sintáctico descendente recursivo en el caso de un análisis exitoso será:

```
CORRECTO: ANALISIS SINTACTICO
```

Para el caso de que el analizador sintáctico encuentre un error el mismo puede ser léxico o sintáctico. En el caso de que sea un error **léxico**, el analizador léxico reportará el error tal y como lo hacía en la etapa 1:

```
ERROR: LEXICO
```

```
| NUMERO DE LÍNEA (NUMERO DE COLUMNA) | DESCRIPCION: |
```

Para el caso de que se trate de un error **sintáctico**, se deberá reportar el error de la siguiente manera, sin excepción:

```
ERROR: SINTACTICO
```

```
| NUMERO DE LÍNEA (NUMERO DE COLUMNA) | DESCRIPCION: |
```

Tenga en cuenta que la descripción debe ayudar al usuario del compilador a entender de qué error se trata. Por ejemplo, **Se esperaba }, se encontró)**

Sobre la documentación de la Etapa Análisis Sintáctico

- Tenga en cuenta la utilización de colores para mejorar la legibilidad de la gramática.
- El texto deberá reflejarse el proceso de transformación de la EBNF de **tinyS** para la implementación de un Analizador Sintáctico Descendente Predictivo Recursivo conforme a lo visto en clase.
- Deberá incluirse la gramática en notación BNF sin extensiones (superíndices +, * y ?).
- Deberá incluirse el resultado de la eliminación de la recursividad a izquierda y la factorización, que genere el mismo lenguaje que la gramática original. Notar que los aspectos regulares del lenguaje tratados durante el análisis léxico no deben formar parte de la BNF factorizada y no recursiva a izquierda.
- Es obligatorio que el texto refleje los pasos que se realizaron para obtener la gramática definitiva. No se evaluará en caso contrario y será desaprobado.
- En el texto deberá constar claramente los errores sintácticos que el analizador es capaz de detectar.

7 Análisis Semántico - Chequeo de Declaraciones

- Se debe implementar la parte del Chequeo de Declaraciones del **Analizador Semántico** de dos pasadas para **tinyS**.
- El *manejo de errores* del analizador semántico deberá realizarse utilizando excepciones. Deberá reportarse un mensaje adecuado a la naturaleza del error e indicar en que número de línea del fuente **tinyS** se produjo el mismo (fijarse que se debe guardar adecuadamente ésta información y dónde).

El formato en caso de error de lo que se deberá mostrar será:

```
ERROR: SEMANTICO - DECLARACIONES  
| NÚMERO DE LINEA (NUMERO DE COLUMNA) | DESCRIPCION: |
```

En caso de que sea exitoso deberá mostrar por pantalla:

```
CORRECTO: SEMANTICO - DECLARACIONES
```

- La política ante la detección de un error es abortar el análisis semántico. No se implementarán técnicas de recuperación ante errores.

La sintaxis de invocación deberá respetar la siguiente convención sin excepción:

```
java -jar etapa3.jar <ARCHIVO_FUENTE>.s
```

donde **etapa3.jar** es el nombre del ejecutable cada cual elige el nombre apropiado que le guste, **<ARCHIVO_FUENTE>.s** es el archivo a analizar las declaraciones.

Si el chequeo de declaraciones es exitoso la salida es el archivo **<ARCHIVO_FUENTE>.ts.json** con la estructura de la tabla de símbolos, consignando las clases que están definidas en la misma. Para cada clase, además se debe informar de quién hereda, sus atributos, sus métodos y constructor con su correspondientes parámetros formales y retorno para el caso de los métodos.

```
>java -jar etapa3.jar prueba.s
```

El formato del archivo de salida debe seguir la estructura de un archivo **json**.

Recordar que el uso de paquetes no propios de java está prohibido. Tengan en cuenta cómo y cuándo realizarán la construcción del archivo **json**.

- En el texto del informe deberá incluirse la especificación del **Esquema de Traducción** (EDT) construido a partir de la gramática factorizada y sin recursión a izquierda, entregada en la segunda etapa del proyecto. El esquema de traducción deberá especificar todas acciones destinadas a la construcción de la Tabla de Símbolos (TS), las mismas pueden especificarse simplemente en términos de intuiciones que llevarán a la posterior implementación.
- Para la realización del EDT es necesaria la especificación de la Tabla de Símbolos del compilador adoptándose la estrategia y las estructuras presentadas en clase.
- El informe deberá incluir una descripción (Diagrama de Clases) de la Tabla de Símbolos: las estructuras usadas y los métodos que provee. Para cada identificador, dependiendo de la entidad asociada, en la tabla de símbolos se deberá almacenar cierta información necesaria durante el proceso de compilación. Para ello deberá definirse una jerarquía de clases que modele las posibles entradas asociadas a cada identificador del lenguaje. En las instancias de las clases de dicha jerarquía deberá indicarse la información que se guarda para cada tipo de entidad.
- Sobre la definición de la Tabla de Símbolos se deberán realizar y documentar todos los chequeos semánticos de `tinyS`. Se debe explicar esquemáticamente las estrategias utilizadas para cada chequeo particular, indicando el contexto en que se utilizan.
- Toda decisión de diseño que se aparte de la definición del lenguaje `tinyS`, o que se tome a partir de cuestiones libradas a la implementación del lenguaje, deberá documentarse en el texto del informe final.

8 Análisis Semántico - Chequeo de Sentencias

- Deberá reportarse un mensaje adecuado a la naturaleza del error e indicar en que número de línea del fuente `tinyS` se produjo el mismo (fijarse que se debe guardar adecuadamente ésta información y dónde).
- La política ante la detección de un error es abortar el análisis semántico. No se implementarán técnicas de recuperación ante errores.

La sintaxis de invocación deberá respetar la siguiente convención sin excepción:

```
java -jar etapa4.jar <ARCH_FUENTE>.s
```

donde `etapa4.jar` es el nombre del ejecutable, `<ARCH_FUENTE>.s` es el archivo a analizar las declaraciones. Si el Análisis Semántico es exitoso se deberá en el `<ARCH_FUENTE>.ts.json` la estructura de la tabla de símbolos, consignando las clases que están definidas en la misma según lo entregada en la Etapa 3. En el archivo `<ARCH_FUENTE>.ast.json` se deberá imprimir la estructura arbórea de los Árboles Sintácticos Abstractos.

El archivo `<ARCH_FUENTE>.ast.json` tendrá información con respecto a la clases declaradas, los métodos y sus sentencias. Además de la información para las clases base y predefinidas, cuya estructura es evaluada.

- Para cada clase se deberá imprimir su nombre.
- Imprimir cada uno de los AST correspondientes a cada sentencia de cada método respetando el orden dentro del método y el tipo de nodo. La estructura debe ser mostrada como un recorrido de cada uno de los ASTs correspondientes a las sentencias permitiendo observar la estructura arbórea de los AST.

- En el texto del informe se deberá incluir una descripción (Diagrama de Clases) de los AST: las estructuras usadas y los métodos que provee. Para cada tipo de nodo deberá definirse una jerarquía de clases que los modele, indicando para cada uno la información que se almacena. Sobre las definiciones del AST se deberán realizar y documentar todos los chequeos semánticos de **tinyS**. Se debe *explicar esquemáticamente las estrategias utilizadas para cada chequeo particular, indicando el contexto en que se utilizan*.
- Decisiones de Diseño: Toda decisión de diseño que se aparte de la definición del lenguaje **tinyS**, o que se tome a partir de cuestiones libradas a la implementación del lenguaje, deberá documentarse en el informe. Se debe describir además el formato con el que se imprimen en el archivo los ASTs y brindar algunos ejemplos. Esto es condición obligatoria para la aprobación de este etapa.
- Casos de Test: Para cada caso de test construido por la comisión debe documentarse **en el informe** si es exitoso o no, su propósito y el resultado esperado.

9 Generación de Código Intermedio

Esta sería la última etapa del proyecto y la que generará el compilador de **tinyS**.

La sintaxis de invocación deberá respetar la siguiente convención sin excepción:

```
java -jar tinyS.jar <ARCH_FUENTE>.s
```

donde **tinyS.jar** es el nombre del ejecutable, **<ARCH_FUENTE>.s** es el archivo a analizar las declaraciones. Si la compilación es exitosa deberá generar los archivos: **<ARCH_FUENTE>.ts.json**, **<ARCH_FUENTE>.ast.json** y **<ARCH_FUENTE>.asm** en el mismo directorio del **.s**. El archivo **.asm** debe ser capaz de ser ejecutado en los simuladores de MIPS, MARS, QtSpim o en Visual Studio Code.

Se deberá incluir información sobre:

- Cuál es la estructura de un Registro de Activación
- Qué almacenan en el Registro de Activación
- Cómo acceden a un parámetro de un método
- Cómo acceden a una variable de un método
- Cómo y dónde retornan un valor en un método
- Qué datos guardar en la Stack antes de la llamada a un método
- Qué datos guardan al principio de un método cuando este es llamado
- Qué y cómo hacen el regreso de una llamada a un método.
- Cómo crean objetos de un struct
- Cómo acceden a los atributos de los objetos instanciados
- Cómo acceden a los métodos de los objetos instanciados

También deberá contener información sobre las clases construidas para el manejo de la generación de código y cualquier otra decisión de diseño que sirva para entender cómo se genera código MIPS a partir del archivo fuente.